Informe de Desarrollo

Desafío II – UdeAWorldCup
Informática II

Nicola Anacona

Andres Maca

Universidad de Antioquia

2026

1. Introducción

El proyecto **UdeAWorldCup** consiste en una simulación del Mundial de Fútbol 2026 desarrollada en C++, el sistema representa selecciones, jugadores, grupos, partidos, fechas, estadísticas históricas y registros de rendimiento por partido. tambien carga información desde archivos CSV, organiza los equipos en bombos, realiza el sorteo de grupos, simula partidos, calcula clasificados y ejecuta las fases finales hasta obtener campeón, subcampeón, tercer y cuarto puesto.

El desarrollo se planteó con programación orientada a objetos, separación entre archivos `.h` y `.cpp`, uso de arreglos dinámicos, punteros y cadenas tipo `char*`. Una parte importante del trabajo fue definir la responsabilidad de cada clase y el manejo correcto de memoria, evitando copias innecesarias, fugas de memoria o dobles liberaciones.

2. Análisis del problema y alternativa de solución propuesta

El problema central consiste en modelar un torneo compuesto por 48 selecciones y cada selección posee datos propios, como país, confederación, ranking FIFA, director técnico y estadísticas históricas. A su vez, cada equipo tiene una plantilla de jugadores con información individual y estadísticas acumuladas. El torneo se divide en fase de grupos y fases eliminatorias, por lo que el programa debe coordinar varias tareas relacionadas: carga de datos, organización de equipos, generación de partidos, simulación de resultados, actualización de estadísticas y clasificación progresiva.

La fase de grupos está formada por 12 grupos de 4 equipos. En cada grupo se juegan seis partidos, porque todos los equipos se enfrentan entre sí una vez. Luego clasifican los primeros, segundos y mejores terceros para formar los 32 equipos de dieciseisavos y ahi el torneo avanza por eliminación directa hasta la final.

La alternativa de solución propuesta fue construir un sistema modular basado en clases. Esta decisión permite que cada elemento del torneo tenga una responsabilidad clara. Por ejemplo, `Equipo` administra los datos de una selección, `Jugador` guarda la información individual de cada futbolista, `Grupo` organiza equipos y calcula posiciones, `Partido` simula un enfrentamiento, y `Torneo` coordina el flujo general.

Para reducir el consumo de memoria, se decidió que estructuras como grupos, bombos, clasificados y top 4 no almacenen copias completas de equipos, sino punteros a equipos ya existentes. Esto evita duplicación de información y mantiene consistencia en las estadísticas: cuando un equipo cambia sus datos durante la simulación, todas las estructuras que lo referencian observan el mismo objeto actualizado.

Las consideraciones principales de diseño fueron:

- Representar las entidades importantes mediante clases independientes.
- Separar declaraciones e implementaciones en archivos `.h` y `.cpp`.
- Usar arreglos dinámicos para estructuras cuyo tamaño depende del torneo.
- Usar punteros para referenciar equipos sin duplicarlos.

- Definir claramente qué clase es dueña de cada recurso dinámico.
- Cargar los datos desde archivos CSV para separar datos y lógica.
- Medir iteraciones y memoria aproximada mediante `MonitorRecursos`.

3. Diagrama de clase

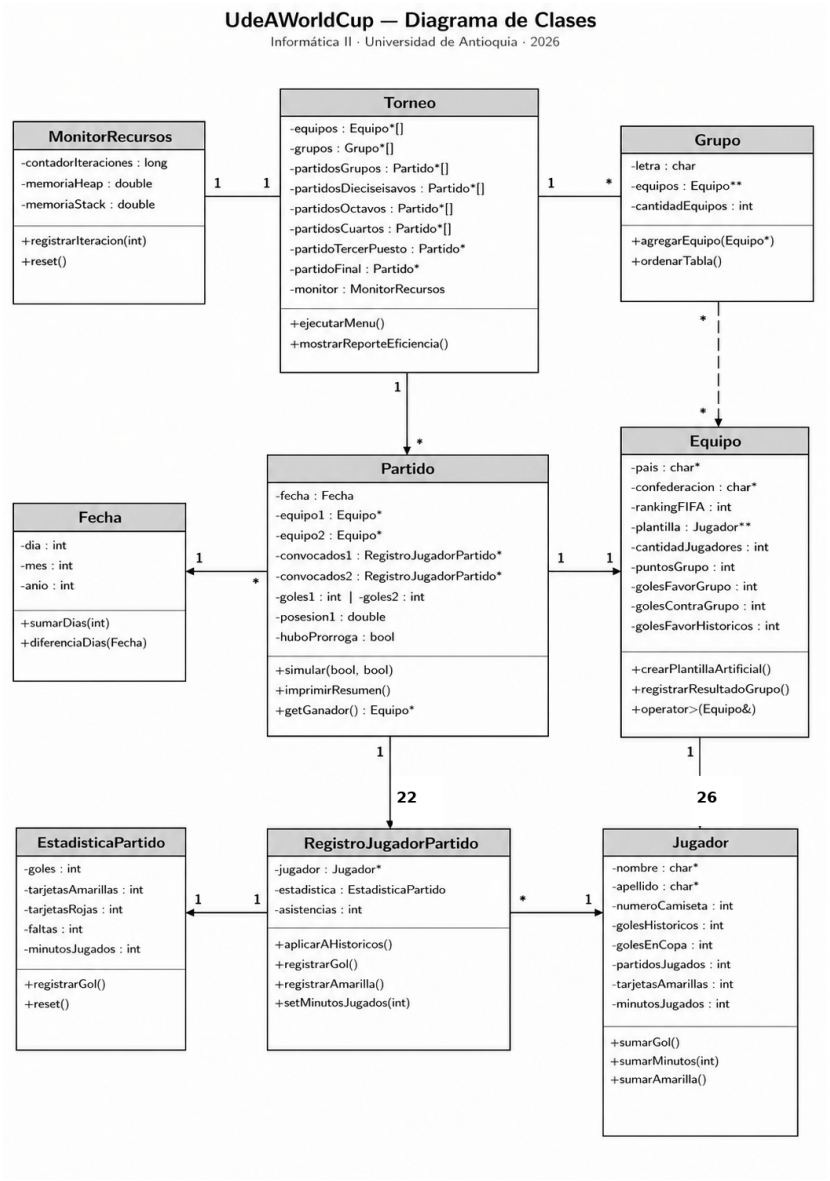

Figura 1: Diagrama de clases de la solución planteada.

El diagrama muestra que `Torneo` funciona como clase coordinadora. Esta clase posee los arreglos principales del sistema y llama a las demás clases para ejecutar tareas específicas. `Grupo` y `Partido` no son dueños de los equipos, solo los referencian mediante punteros. En cambio, `Equipo` sí es dueño de su plantilla de jugadores, y `Jugador` es responsable de sus cadenas dinámicas.

4. Descripción general de las clases

Jugador: representa un jugador de una selección. Almacena nombre, apellido, número de camiseta y estadísticas personales. Como maneja cadenas dinámicas, debe liberar memoria en su destructor y controlar copias correctamente.

Equipo: representa una selección nacional. Guarda país, confederación, ranking, director técnico, estadísticas históricas y plantilla dinámica. Es dueño de su arreglo de jugadores.

EstadisticaHistorica: separa los datos acumulados de rendimiento de una selección, como partidos, goles, victorias, derrotas, empates o puntos.

Grupo: representa un grupo del mundial. Contiene una letra identificadora y punteros a equipos. Ordena la tabla según criterios de clasificación.

Fecha: almacena día, mes y año de los partidos. Permite imprimir fechas y avanzar días dentro del calendario del torneo.

Partido: representa un enfrentamiento entre dos equipos. Administra marcador, fecha, registros de jugadores y actualización de estadísticas del encuentro.

EstadisticaPartido: guarda eventos específicos de un partido, como goles, asistencias, tarjetas, faltas y minutos jugados.

RegistroJugadorPartido: relaciona un jugador con su rendimiento en un partido. Permite registrar primero los datos del encuentro y después aplicarlos al historial del jugador.

MonitorRecursos: cuenta iteraciones y calcula memoria aproximada usada por las estructuras principales del sistema.

5. Algoritmos esenciales debidamente intra-documentados

En esta sección se explican los algoritmos principales en alto nivel. No se copia código fuente, sino que se describe la lógica de los subprogramas cuya solución no es trivial.

5.1. Carga de equipos desde CSV

Propósito: leer el archivo de selecciones clasificadas y crear los objetos `Equipo` que serán usados durante todo el torneo.

Lógica del algoritmo:

1. Abrir el archivo CSV en modo lectura.
2. Leer la primera línea y descartarla si corresponde al encabezado.
3. Leer cada línea restante de manera secuencial.
4. Separar los campos según el delimitador del archivo.
5. Validar que la línea tenga los datos mínimos esperados.
6. Convertir los campos numéricos, como ranking o valores estadísticos.
7. Configurar un objeto `Equipo` con país, confederación, ranking y datos históricos.
8. Guardar el equipo en el arreglo dinámico principal administrado por `Torneo`.

Consideración: la lectura secuencial evita cargar todo el archivo en memoria al mismo tiempo. Además, separar la lectura de datos de la simulación facilita corregir errores del CSV sin modificar la lógica del torneo.

5.2. Creación de jugadores

Propósito: asignar una plantilla a cada equipo para que los partidos puedan registrar goles, minutos y eventos individuales.

Lógica del algoritmo:

1. Recorrer el arreglo de equipos cargados.
2. Para cada equipo, reservar memoria dinámica para su plantilla.
3. Crear los jugadores uno por uno con nombre, apellido, dorsal y estadísticas iniciales.

4. Asociar la plantilla al equipo correspondiente.
5. Dejar la liberación de esa plantilla bajo responsabilidad de la clase `Equipo`.

Consideración: esta tarea incrementa el uso de memoria porque cada selección recibe su propio arreglo de jugadores. Por eso es importante que `Equipo` tenga un destructor adecuado.

5.3. Formación de bombos

Propósito: organizar los equipos según ranking para preparar el sorteo de grupos.
Lógica del algoritmo:

1. Crear una estructura auxiliar con punteros a los equipos ya cargados.
2. Ordenar esos punteros según el ranking FIFA.
3. Dividir el arreglo ordenado en cuatro bombos de tamaño equivalente.
4. Guardar en cada bombo punteros a equipos, no copias completas.

Consideración: aunque podrían usarse métodos de ordenamiento más avanzados, el tamaño del problema es pequeño y fijo. Por eso un algoritmo simple resulta suficiente y más fácil de revisar.

5.4. Sorteo de grupos

Propósito: distribuir los equipos en 12 grupos de 4, respetando restricciones de confederación cuando sea posible.
Lógica del algoritmo:

1. Inicializar los 12 grupos vacíos.
2. Recorrer los bombos y tomar equipos disponibles.
3. Intentar ubicar cada equipo en un grupo con cupo.
4. Verificar que el grupo no incumpla las reglas de confederación.
5. Si el grupo es válido, agregar el puntero del equipo.
6. Si no es válido, intentar con otro grupo.
7. Repetir hasta completar todos los grupos.

Consideración: el sorteo puede bloquearse si una combinación deja grupos imposibles de completar. Por eso se contemplan reintentos o validaciones adicionales para evitar distribuciones inválidas.

5.5. Generación de partidos de fase de grupos

Propósito: crear los seis enfrentamientos necesarios en cada grupo.
Lógica del algoritmo:

1. Recorrer cada grupo.
2. Tomar sus cuatro equipos.
3. Generar las combinaciones sin repetición: equipo 1 contra 2, 1 contra 3, 1 contra 4, 2 contra 3, 2 contra 4 y 3 contra 4.
4. Crear un objeto `Partido` para cada enfrentamiento.
5. Asignar fecha y guardar el partido en el arreglo correspondiente.

Consideración: como son 12 grupos y cada uno produce 6 partidos, la fase de grupos genera 72 partidos.

5.6. Simulación de partidos

Propósito: obtener un marcador, actualizar estadísticas de equipos y registrar eventos individuales.
Lógica del algoritmo:

1. Validar que ambos equipos existan.
2. Preparar registros para jugadores convocados.

3. Calcular una base de rendimiento usando ranking, estadísticas o datos históricos.
4. Generar goles para cada equipo.
5. Si el partido es eliminatorio y hay empate, resolver un ganador.
6. Asignar eventos a jugadores, como goles o tarjetas.
7. Actualizar estadísticas del partido.
8. Aplicar los registros individuales al historial de cada jugador.
9. Actualizar estadísticas históricas de los equipos.

Consideración: en fase de grupos se permiten empates, pero en eliminatorias debe existir un ganador. Por eso el algoritmo cambia ligeramente según la etapa del torneo.

5.7. Clasificación a dieciseisavos

Propósito: seleccionar los 32 equipos que avanzan a fase eliminatoria.
Lógica del algoritmo:

1. Ordenar la tabla de cada grupo por puntos, diferencia de gol y goles a favor.
2. Agregar a clasificados los primeros 12 equipos.
3. Agregar a clasificados los segundos 12 equipos.
4. Guardar los terceros de grupo en una lista auxiliar.
5. Ordenar los terceros con los mismos criterios de rendimiento.
6. Seleccionar los mejores 8 terceros.

Consideración: los clasificados se guardan como punteros a `Equipo`. Esto evita duplicar selecciones completas y mantiene actualizadas sus estadísticas.

5.8. Simulación de fases finales

Propósito: avanzar por eliminación directa hasta definir el campeón.
Lógica del algoritmo:

1. Tomar los 32 clasificados.
2. Simular dieciseisavos y guardar 16 ganadores.
3. Simular octavos y guardar 8 ganadores.
4. Simular cuartos y guardar 4 ganadores.
5. Simular semifinales y separar finalistas de perdedores.
6. Simular partido por tercer puesto.
7. Simular la final.
8. Guardar campeón, subcampeón, tercer y cuarto puesto.

Consideración: cada ronda reduce a la mitad la cantidad de equipos, excepto los partidos finales adicionales. Como no puede haber empate, cada partido eliminatorio debe resolver un ganador.

6. Manejo de memoria y eficiencia

El proyecto usa memoria dinámica en equipos, jugadores, grupos, partidos, clasificados y estructuras auxiliares. Por eso fue necesario establecer propiedad de memoria:

- `Torneo` es dueño del arreglo principal de equipos, grupos, partidos y clasificados.
- `Equipo` es dueño de su plantilla dinámica de jugadores.
- `Jugador` es dueño de sus cadenas dinámicas.

- `Grupo` solo referencia equipos; no debe destruirlos.
- `Partido` solo referencia equipos, pero sí administra sus recursos propios.

La eficiencia se trabajó principalmente evitando copias de objetos grandes. En vez de copiar equipos completos dentro de grupos, bombos o fases finales, se guardan punteros a los equipos originales. Esto reduce memoria y evita inconsistencias.

También se incluyó `MonitorRecursos`, que estima iteraciones y memoria usada. Aunque no reemplaza una medición real del sistema operativo, sí permite comparar etapas del programa y justificar el crecimiento de memoria al crear plantillas, partidos o fases finales.

7. Problemas de desarrollo afrontados

Durante el desarrollo se presentaron varios problemas técnicos:

- **Lectura de archivos CSV:** fue necesario revisar el separador real del archivo y evitar interpretar el encabezado como datos de un equipo.
- **Organización modular:** al manejar muchas clases, fue necesario separar correctamente archivos `.h` y `.cpp`, incluir cabeceras y evitar dependencias circulares.
- **Errores de enlazado:** algunos errores podían aparecer cuando una función estaba declarada pero no implementada, o cuando el archivo fuente no estaba incluido en el proyecto.
- **Manejo de memoria dinámica:** se debía evitar liberar dos veces un mismo recurso. Para esto se definió qué clase era dueña de cada arreglo o cadena.
- **Copias superficiales:** las clases con punteros requieren cuidado al copiar objetos. Si dos objetos apuntan a la misma memoria, sus destructores podrían causar errores.
- **Sorteo de grupos:** la restricción por confederaciones hizo que el sorteo no fuera una simple distribución aleatoria. Fue necesario validar cada asignación.
- **Simulación de eliminatorias:** en grupos puede haber empate, pero en fases finales no. Esto obligó a distinguir la lógica de simulación según la etapa.

8. Evolución de la solución

La solución se construyó de forma progresiva. Primero se implementaron clases base para representar las entidades principales del dominio, como `Jugador`, `Equipo`, `Fecha` y `Grupo`. Después se agregaron las clases estadísticas, como `EstadisticaHistorica`, `EstadisticaPartido` y `RegistroJugadorPartido`. Finalmente, se integró la clase `Torneo` como coordinadora del flujo general.

El desarrollo avanzó en las siguientes etapas:

1. Definición de clases del dominio.
2. Implementación de constructores, destructores y métodos básicos.
3. Carga de equipos desde archivos CSV.
4. Creación de plantillas de jugadores.
5. Formación de bombos y sorteo de grupos.
6. Generación de partidos de fase de grupos.
7. Simulación de partidos y actualización de estadísticas.
8. Clasificación a dieciseisavos.
9. Simulación de fases finales.
10. Incorporación de reportes y medición de recursos.

Esta evolución permitió probar el programa por partes. Primero se verificó la carga de datos, luego la creación de grupos, después la generación de partidos y finalmente la simulación completa del torneo.

9. Consideraciones de implementación

Para implementar correctamente la solución se deben tener en cuenta los siguientes aspectos:

- Mantener una clase por archivo de cabecera e implementación cuando sea posible.
- Incluir todos los archivos `.cpp` en el proyecto para evitar errores de compilación o enlazado.
- Usar `delete[]` para arreglos dinámicos y `delete` para objetos individuales.
- No destruir desde `Grupo` o `Partido` equipos que pertenecen a `Torneo`.
- Inicializar punteros antes de usarlos y dejarlos en `nullptr` después de liberar memoria importante.
- Validar los archivos antes de cargar datos para evitar equipos incompletos.
- Separar la lógica de simulación de la lógica de presentación en consola.
- Documentar las funciones complejas con propósito, entradas, proceso y salida.

Una mejora posible sería usar máscaras de bits para representar estados del torneo, por ejemplo, equipos cargados, jugadores creados, grupos sorteados o fases simuladas. Esto permitiría compactar varios estados booleanos en un solo byte. Sin embargo, en la versión actual el centro del proyecto está en la orientación a objetos, memoria dinámica, archivos y simulación del torneo.

10. Conclusiones

El desarrollo de UdeAWorldCup permitió aplicar conceptos fundamentales de C++, especialmente programación orientada a objetos, modularidad, punteros, arreglos dinámicos, lectura de archivos y administración manual de memoria.

La decisión de usar punteros para referenciar equipos en grupos, bombos y clasificados fue importante para reducir copias innecesarias y mantener consistencia en los datos. Además, definir propiedad de memoria permitió disminuir riesgos de fugas o dobles liberaciones.

Aunque el modelo de simulación es una aproximación y puede mejorarse con reglas más realistas, la solución cumple el objetivo académico: construir un sistema modular en C++ que integre clases, archivos, memoria dinámica, algoritmos de organización y simulación de una competencia deportiva.

11. Referencias

1. Enunciado del Desafío II, Informática II, Universidad de Antioquia, 2026.
2. Material de clase sobre programación orientada a objetos en C++.
3. Código fuente del proyecto UdeAWorldCup.